

A PROJECT REPORT ON

**Test Case Generation From User Requirements
Using LLM And Generative AI**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Name	Exam No
Ayush Bulbule	B150054328
Soumya Garg	B150054396
Awadhoot Khutwad	B150054450
Sumit Suryawanshi	B150054606



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Test Case Generation From User Requirements Using LLM And Generative AI

Submitted by

Name	Exam No
Ayush Bulbule	B150054328
Soumya Garg	B150054396
Awadhoot Khutwad	B150054450
Sumit Suryawanshi	B150054606

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. M. S. Wakode** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. M. S. Wakode
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Test Case Generation From User Requirements Using LLM And Generative AI**’.*

*We would like to take this opportunity to thank **Prof. M. S. Wakode** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support, suggestions.*

*In the end our special thanks to **Nikhil Karkhanis**, Sarvatra Technologies and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Ayush Bulbule
Soumya Garg
Awadhoot Khutwad
Sumit Suryawanshi
(B.E. Computer Engg.)

ABSTRACT

Current software testing platforms face significant limitations, including the manual creation of test cases—a process that is time-consuming, error-prone, and often misses critical edge or boundary conditions. These gaps can result in incomplete coverage and undetected bugs, ultimately affecting software quality. Moreover, the lack of automated tools capable of handling complex validation logic or visual inputs, such as forms or UI elements, further compounds the problem. To address these challenges, we present an AI-powered testing platform that leverages multimodal Large Language Models (LLMs) and Generative AI. This solution automates the generation of comprehensive test cases from both textual and visual inputs, enabling the detection of functional, edge, and boundary cases with high accuracy. The system can analyze code logic, validate conditions, and generate scripts for various input-output scenarios—significantly reducing manual effort and accelerating the development lifecycle. Furthermore, by supporting image-based test case generation, the platform bridges the gap between functional and visual validation. This approach enables continuous, scalable, and precise testing, empowering developers and testers to achieve higher coverage and faster bug detection with reduced overhead.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	3
1.3	Problem Definition and Objectives	3
1.3.1	Problem Definition	3
1.3.2	Objectives	4
1.4	Project Scope & Limitations	4
1.5	Methodology of Problem Solving	5
1.5.1	Test Case Structuring	6
1.5.2	Validation and Optimization	6
1.5.3	Output Generation	6
1.6	Conclusion	6
2	Literature Survey	7
3	Software Requirements Specification	10
3.1	Assumptions and Dependencies	11
3.1.1	Project Assumptions	11
3.1.2	System Dependencies	11
3.1.3	User Classes and Characteristics	12
3.1.4	Dependency Management	12
3.1.5	Risk Factors and Mitigation	13
3.2	Functional Requirements	13
3.2.1	User Authentication System	13
3.2.2	Multi-Format Input Processing	13
3.2.3	AI Analysis Capabilities	14
3.2.4	Results Generation	14
3.2.5	Interactive User Interface	14
3.2.6	Data Management	15
3.2.7	External Integration	15
3.3	External Interface Requirements	16

3.3.1	User Interfaces	16
3.3.2	Hardware Interfaces	16
3.3.3	Software Interfaces	16
3.3.4	Communication Interfaces	17
3.4	Nonfunctional Requirements	18
3.4.1	Performance Requirements	18
3.4.2	Safety Requirements	18
3.4.3	Security Requirements	18
3.4.4	Software Quality Attributes	18
3.5	System Requirements	19
3.5.1	Database Requirements	19
3.5.2	Software Requirements (Platform Choice)	19
3.5.3	Hardware Requirements	19
3.6	Analysis Models: SDLC Model to be Applied	20
4	System Design	21
4.1	System Architecture	22
4.2	Data Flow Diagrams	22
4.3	Entity Relationship Diagram	23
4.4	UML Diagrams	25
4.5	Sequence Diagram	27
5	Project Plan	29
5.1	Project Estimate	30
5.1.1	Reconciled Estimates	30
5.1.2	Project Resources	31
5.2	Risk Management	32
5.2.1	Risk Identification	32
5.2.2	Risk Analysis	32
5.2.3	Overview of Risk Mitigation, Monitoring, Management	33
5.3	Project Schedule	34
5.3.1	Project Task Set	34
5.3.2	Timeline Chart	35
5.4	Team Organization	35
5.4.1	Team Structure	35
5.4.2	Management Reporting and Communication	35
6	Project Implementation	36
6.1	Overview of Project Modules	37
6.2	Tools and Technologies Used	37
6.3	Algorithm Details	38

6.3.1	Algorithm 1: Test Case Generation Using LLMs	38
6.3.2	Algorithm 2: Flow Chart Generation Using Gemini LLM	40
6.3.3	Algorithm 3: Vision-Based Test Case Generation	41
6.3.4	Algorithm 4: Conversational Agent for Test Case Query- ing	42
7	Software Testing	45
7.1	Type of Testing	46
7.1.1	Backend Testing	46
7.1.2	Frontend Testing	47
7.1.3	Integration Testing	47
7.2	Test cases & Test Results	47
8	Results	48
8.1	Outcomes	49
8.2	Screenshots	50
8.2.1	Text Input and Output	50
8.2.2	Image Input and Output	51
9	Conclusions	52
9.1	Conclusions	53
9.2	Future Work	53
9.3	Applications	54
10	Appendix	55
10.1	Appendix A: Feasibility Study	56
10.1.1	Technical Feasibility	56
10.1.2	Operational Feasibility	56
10.1.3	Economic Feasibility	57
10.2	Appendix B	58
10.2.1	Survey Paper: Conference Information	58
10.2.2	Implementation Paper: Conference Information	59
10.3	Appendix C	60
10.3.1	Sponsorship Certificate	60
10.4	Appendix D	61
11	References	62

List of Figures

4.1	Data Flow Diagram	22
4.2	Data Flow Diagram	22
4.3	ER Diagram	23
4.4	Use Case Diagram	24
4.5	Activity Diagram	25
4.6	Activity Diagram	26
4.7	Sequence Diagram	27
5.1	Summarized Timeline Chart	35
8.1	Input .md File ATM	50
8.2	Generated Test Cases	50
8.3	Test Flow	50
8.4	Input Image 3	51
8.5	Output Image 3a	51
10.1	Certificate of Authorship – ERCICAM 2025 (Scopus Indexed)	58
10.2	Screenshot of the Paper Submission	59
10.3	Sponsorship Certificate from Sarvatra Technologies	60
10.4	Plagiarism Report	61

CHAPTER 1

INTRODUCTION

1.1 Overview

Software testing is vital for evaluating software quality and directly influences project cost and timely delivery. A system's effectiveness is often measured by customer satisfaction, making it essential to ensure it meets specified requirements. Software testing is defined as "a process of exercising or evaluating a system or system components by manual or automated means to verify that it satisfies specified requirements."

Test cases are designed to validate that all functional requirements have been accurately implemented. They serve as a key mechanism to confirm the system aligns with specifications and meets customer expectations. Traditionally, test case generation has been a manual, time-consuming, and error-prone task. However, the rise of Large Language Models (LLMs) has introduced new possibilities for automating this process. With their advanced natural language processing capabilities, LLMs can bridge the gap between abstract user requirements and executable test cases, significantly improving the efficiency and accuracy of software testing.

To address these challenges, we developed **TestGen.AI**, an AI-powered tool that automates test case generation from natural language specifications. It supports leading LLMs such as OpenAI's GPT and Google's Gemini, allowing users to select models based on their preferences and testing requirements. TestGen.AI accepts two types of input:

- **Textual inputs**, like a document describing the system, based on which test cases are generated or even API documentation.
- **Image-based inputs**, such as UML diagrams or sequence diagrams, from which it extracts logic to generate relevant test cases.

Beyond test case generation, **TestGen.AI** offers several advanced capabilities that enhance the overall software testing workflow. It supports **automatic JMeter script (.jmx) generation** for load and performance testing, eliminating the need for manual configuration within JMeter. The tool can also generate **sample database queries** aligned with test cases, based on the provided database schema, to assist in validating backend data interactions. Furthermore, it provides **end-to-end test case workflow generation**, enabling testers to follow a structured and coherent sequence of actions. Additionally, **automated sequence diagram creation** is supported, offering visual representations of system interactions and workflows derived directly from the input specifications.

A key strength of **TestGen.AI** is its **intuitive, easy-to-use interface**, which streamlines the testing process. Users can convert raw speci-

fications into actionable test artifacts within minutes—drastically reducing reliance on manual efforts, minimizing errors, and boosting productivity and consistency.

1.2 Motivation

The motivation behind this project stems from several key challenges in manual test case generation. First, the process of manually creating test cases from requirement documents is both time-consuming and error-prone. In contrast, an AI-based tool significantly accelerates this process, allowing for faster and more efficient test case generation.

Furthermore, as software projects grow, so does the number of requirements and corresponding test cases. The scalability provided by automation enables teams to uphold high testing standards without overwhelming the quality assurance (QA) team, allowing the process to adapt effortlessly to increasing project complexity.

Manual test case creation is also susceptible to human error, with potential oversights or missed scenarios. An AI-powered tool mitigates these risks by ensuring that edge cases and other critical scenarios based on the requirements are systematically captured, enhancing the thoroughness of the testing process.

Lastly, in Agile or DevOps workflows, where frequent changes in requirements are common, keeping test cases up-to-date poses an additional challenge. An AI-based tool can dynamically generate and update test cases in alignment with the latest requirements, enabling continuous testing in fast-paced development environments.

1.3 Problem Definition and Objectives

1.3.1 Problem Definition

Manual test case creation is often inefficient and prone to human error due to its repetitive and detailed nature. Testers must thoroughly review user requirements and interpret them to generate test cases, which can lead to overlooked scenarios, incomplete coverage, and delays in the software development lifecycle. As software systems grow in complexity, the traditional manual approach becomes increasingly insufficient in ensuring thorough testing and high-quality assurance.

To address these challenges, the proposed solution, **TestGen.AI**, harnesses the power of advanced AI techniques, specifically Natural Language Processing (NLP) and Large Language Models (LLMs). By automating the test case generation process, **TestGen.AI** aims to streamline test creation, reduce human error, and improve efficiency. The AI-powered system analyzes user requirements documents, API specifications, and system diagrams, extracting critical information to generate structured, comprehensive, and relevant test cases. These generated test cases align with specified criteria, ensuring thorough coverage and enhancing the overall quality assurance process.

1.3.2 Objectives

- Finalize the system requirements, including detailed specifications for the AI-driven test case generation process, and create a documented requirements specification along with the finalized system design, incorporating block diagrams and the flow of data between modules.
- Develop and implement the core modules for test case generation, including integration with AI models for automatic test case creation from textual and diagrammatic inputs. Build the modules for JMeter script generation, database query generation, and workflow visualization. Integrate all system modules to ensure smooth operation and optimize the performance of the test case generation process.
- Conduct user testing of the system, incorporating feedback to enhance functionality, usability, and accuracy. Based on testing outcomes, refine the system for continuous improvement, and implement necessary fixes or optimizations.

1.4 Project Scope & Limitations

The scope of this project focuses on automating the creation of test cases using advanced AI techniques, specifically leveraging Large Language Models (LLMs) and Natural Language Processing (NLP). The primary objective is to enhance software testing efficiency by automatically generating comprehensive test cases from user requirements documents, API documentation, and system diagrams such as UML and sequence diagrams. This solution aims to reduce manual effort, eliminate human errors, and improve test coverage by capturing various test scenarios, including edge cases. **TestGen.AI** supports diverse input types, allowing testers to generate test cases based on

textual descriptions, API documentation, or visual models, providing greater flexibility in test creation.

The project also extends its capabilities to generate **JMeter scripts** for performance and load testing, eliminating the need for manual JMeter configuration. Additionally, it can generate **sample database queries** aligned with test cases to verify backend interactions. The tool further includes **end-to-end test case workflow generation** to guide testers through the testing process and ensure comprehensive test execution. The AI-driven platform also facilitates **automated test case workflow creation**, enabling a visual graphical representation of all the test cases.

In terms of scope, **TestGen.AI** can be applied across various industries and software domains, offering potential customization through fine-tuning of the AI models for specific needs. It also aims for seamless integration with continuous testing frameworks, supporting automated testing in the software development pipeline. The system supports multiple languages, making it globally applicable for diverse projects. Moreover, it allows testers to customize test case generation based on parameters such as test type (functional, boundary, performance) and priority level.

However, there are certain limitations associated with the project. While **TestGen.AI** handles functional test cases effectively, generating non-functional test cases—such as those related to performance, security, or stress testing—might require further specialized training. Performance concerns may arise as LLMs require significant computational resources, particularly when processing large documents or complex diagrams. Finally, while **TestGen.AI** can streamline the generation of test cases, human intervention may still be necessary to validate complex, domain-specific requirements or when dealing with edge cases not well represented in the training data. Ethical and legal concerns may also arise in industries like healthcare or finance, where inaccurate or incomplete test cases could have significant consequences.

1.5 Methodology of Problem Solving

The LLM (e.g., GPT) analyzes the requirement document along with the retrieved data to extract:

- **Functional Requirements:** The functionalities that need to be tested.
- **Conditions:** Preconditions, postconditions, and operational constraints.
- **Expected Outcomes:** The system's expected behavior for each scenario.

1.5.1 Test Case Structuring

Based on the enriched understanding of the requirements, the LLM generates test cases in an industry-accepted format of test case description:

- **Test Case ID:** A unique identifier for each test case.
- **Test Description:** A brief explanation of the test case.
- **Preconditions and Steps:** Detailed steps for execution.
- **Expected Outcome:** The result confirming correct functionality.

1.5.2 Validation and Optimization

The generated test cases can be reviewed and validated by the user for accuracy. If modifications or additional scenarios are needed, the user can interact with the integrated chat interface to prompt the LLM for refined or updated test cases based on their suggestions. Additionally, the platform provides an interactive UI that allows users to visualize the complete test case workflow, offering a clear overview of the testing process. For each test case, users can also access a detailed breakdown of the individual testing steps, ensuring clarity and thorough understanding.

1.5.3 Output Generation

The validated test cases are exported in a suitable CSV format and integrated into test management tools like Jira, TestRail, or Azure DevOps.

1.6 Conclusion

TestGen.AI utilizes AI and LLMs to automate test case generation, significantly improving the efficiency and accuracy of the testing process. By supporting various input types—ranging from natural language descriptions to UML diagrams—it provides developers and QA engineers with a versatile tool that adapts to their workflow. The tool's advanced features, such as JMeter script generation and database query creation, make it a comprehensive solution for both functional and performance testing. As a result, TestGen.AI helps teams achieve higher test coverage with minimal manual effort, ultimately driving better software quality and faster release cycles.

CHAPTER 2

LITERATURE SURVEY

Several studies have explored the use of Large Language Models (LLMs) in automated test case generation. In recent research, LLMs have shown promise for automating the generation of test cases, though challenges such as model reliability and the quality of generated test cases persist. We summarize key studies in this area below.

- **A Tool for Test Case Scenarios Generation using Large Language Models [1]:** Abdul Malik Sami and Zeeshan Rasheed (Tampere University) introduced a web-based tool that leverages LLMs, specifically OpenAI's models, to generate test case scenarios from user requirements. This tool prioritizes test case scenarios from user stories, thus streamlining the software testing process and reducing manual effort. However, the authors highlight challenges with "LLM hallucinations," where outputs may not align with input data, impacting the reliability of automated test case generation. They suggest that integrating Retrieval-Augmented Generation (RAG) techniques could help mitigate these issues. Furthermore, content analysis automation in generated test cases remains in its early stages and requires further refinement for accuracy and robustness.
- **Automated Unit Test Improvement using Large Language Models at Meta [2]:** N. Alshahwan et al. describe Meta's automated unit test generation tool, which uses LLMs to reduce manual testing effort by generating additional test cases for missed edge cases. The tool showed a 73% acceptance rate for LLM-generated tests by engineers for production use. However, some generated tests lacked essential assertions or focused on trivial methods, limiting their effectiveness. Additionally, only 25% of test cases contributed to increased test coverage, indicating limited success in expanding coverage. The tool primarily extended human-written tests rather than creating entirely new ones.
- **Large Language Models as Test Case Generators: Performance Evaluation and Enhancement [3]:** Kefan Li and Yuan Yuan (Beihang University) proposed the TestChain framework, a multi-agent system designed to improve LLMs' performance in test case generation. TestChain consists of two agents: a Designer Agent that generates diverse test inputs and a Calculator Agent that maps inputs to outputs using a Python interpreter. This approach reduces complexity and improves accuracy in test case generation. However, the TestChain framework is limited to robust models like GPT-3.5 and GPT-4, making it less applicable for weaker models. The framework's evaluation

is based on specific datasets (HumanEval and LeetCode-hard), which may not represent the diversity of real-world programming tasks.

- **Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation** [4]: Benjamin Steenhoeck, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy (Microsoft, USA) investigate reinforcement learning (RL) as a method to improve the quality of LLM-generated test cases. Their study identified common issues in generated tests, such as test smells, missing assertions, and improper method calls. By applying RL methods based on static quality metrics, they achieved a 21.4% improvement in quality regarding best practices like method invocation and assertion usage. The RL-optimized model outperformed GPT-4 on four out of seven quality metrics. However, challenges remain, such as creating descriptive test documentation, and instabilities during RL training that sometimes reduced the diversity of generated test cases.

Summary of Literature Survey: These studies demonstrate significant advancements in using LLMs for test case generation, with tools showing potential to automate testing processes and improve coverage. However, challenges such as LLM hallucinations, limited coverage expansion, and the need for RL training stabilization remain. Integrating RAG techniques and refining RL models are recommended to improve the accuracy, reliability, and quality of automated test cases generated by LLMs.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Project Assumptions

3.1.1.1 Technical Assumptions

- Users have access to modern web browsers and sufficient network bandwidth (2+ Mbps)
- Input data follows standard formatting conventions for each supported type (text, images, JMX, DB schemas)
- AI components can effectively parse domain-specific terminology and identify relationships
- Authentication mechanisms provide adequate security for project information

3.1.1.2 Business Assumptions

- Developers and testers will adopt the tool within their existing workflows
- Organizational infrastructure supports deployment and maintenance
- Core use cases address the majority of user needs across testing and development

3.1.2 System Dependencies

3.1.2.1 Technical Dependencies

- Cloud computing resources for AI processing and data storage
- AI frameworks (TensorFlow/PyTorch) and database systems
- Authentication services and file processing libraries
- Front-end frameworks for responsive UI implementation

3.1.2.2 Integration Dependencies

- APIs for development tools (IDEs, version control)
- Test management system integration points
- CI/CD pipeline compatibility
- Data sources for AI model training and validation

3.1.3 User Classes and Characteristics

3.1.3.1 Primary User Classes

Software Developers Technical users focused on code quality and TDD approaches. They upload code and requirements for analysis and generate test scenarios.

Quality Assurance Engineers Testing experts focused on test coverage and edge case identification. They analyze requirements and export test scenarios.

Project Managers Oversight roles focused on project quality metrics, risk management, and progress tracking.

3.1.3.2 Secondary User Classes

System Administrators Technical staff responsible for system configuration, performance, and user management.

Business Analysts Requirements-focused users who validate system interpretations of requirements.

3.1.4 Dependency Management

The project implements version control with strict compatibility testing, fallback mechanisms for service disruptions, monitoring systems for dependency health, and comprehensive documentation for administrators and users to ensure reliable operation while minimizing external dependency risks.

3.1.5 Risk Factors and Mitigation

Risk Factor	Description	Mitigation Strategy
AI Accuracy	AI may not achieve sufficient accuracy for complex inputs	Implement confidence scoring and human review options
Integration Issues	Existing toolchains may present compatibility challenges	Develop flexible API adapters and early integration testing
User Adoption	Learning curve may impede initial usage	Comprehensive onboarding and phased feature introduction
Data Security	Sensitive code handling may raise concerns	Robust data policies and on-premises deployment options
System Scalability	Performance degradation under high usage	Design for horizontal scaling with performance monitoring

Table 3.1: Key Risk Factors and Mitigation Strategies

3.2 Functional Requirements

3.2.1 User Authentication System

The system shall provide secure login functionality to authenticate users with different roles. It must support credential validation, maintain secure sessions, and distinguish between developer and tester access privileges.

- FR-1.1: The system shall authenticate users through username and password.
- FR-1.2: The system shall differentiate between developer and tester user roles.
- FR-1.3: The system shall maintain secure user sessions throughout workflow.

3.2.2 Multi-Format Input Processing

The system shall accept and process multiple input types including text documents, images, JMX files, database structures, and test flows. It shall validate these inputs and extract relevant entities and relationships.

- FR-2.1: The system shall support five input types (text, image, JMX, database, test flow).
- FR-2.2: The system shall validate uploaded file formats and content.
- FR-2.3: The system shall extract entities and relationships from inputs.

3.2.3 AI Analysis Capabilities

The system shall provide AI-driven analysis that adapts to user type, generating TDD scenarios for developers and identifying edge cases for testers. Analysis results should be interactive and refinable.

- FR-3.1: The system shall adapt analysis based on user type (developer/tester).
- FR-3.2: The system shall generate TDD scenarios for developers.
- FR-3.3: The system shall identify test flow gaps and edge cases for testers.

3.2.4 Results Generation

The system shall format results according to standardized templates and prepare downloadable files in user-preferred formats with appropriate metadata.

- FR-4.1: The system shall structure outputs in standardized formats.
- FR-4.2: The system shall prepare downloadable result files.
- FR-4.3: The system shall include generation metadata with all outputs.

3.2.5 Interactive User Interface

The system shall provide an interactive dashboard where users can view results and perform actions including refinement requests, exports, saving, and feedback submission.

- FR-5.1: The system shall display analysis results in an interactive dashboard.
- FR-5.2: The system shall support result refinement through AI chat.
- FR-5.3: The system shall enable export, save, and feedback actions.

3.2.6 Data Management

The system shall store user data, analysis results, and feedback securely, implementing appropriate retention policies and access controls.

- FR-6.1: The system shall store user profiles and analysis history.
- FR-6.2: The system shall save analysis results with access controls.
- FR-6.3: The system shall record user feedback for AI improvement.

3.2.7 External Integration

The system shall provide APIs and export formats that enable integration with external development and testing tools.

- FR-7.1: The system shall provide API endpoints for external tool integration.
- FR-7.2: The system shall support standard export formats for testing tools.
- FR-7.3: The system shall enable imports from development environments.

3.3 External Interface Requirements

3.3.1 User Interfaces

The TestGen.AI platform provides a modern, intuitive, and responsive web interface designed for QA engineers, developers, and business analysts. Users can interact with the platform in two primary ways:

- **File Upload Interface:** Allows users to upload requirement documents, API specifications, sequence diagrams, and PlantUML files through a simple drag-and-drop or select-from-disk interface.
- **AI Chat Interface:** A conversational interface where users can ask questions, clarify ambiguities, and iteratively refine test cases generated by the system.
- **Test Case Viewer:** Displays generated test cases in a structured format. Supports export to JSON, CSV, or text.
- **Feedback Module:** Allows users to provide ratings or comments on the generated test cases for future improvement.

The interface follows accessibility guidelines and supports responsive design for use on desktops, tablets, and mobile devices.

3.3.2 Hardware Interfaces

- **Client-Side:** Any modern device (laptop, desktop, tablet) capable of running a web browser (Chrome, Firefox, Edge).
- **Server-Side:** Hosted on cloud infrastructure (e.g., AWS EC2/Containers), requiring:
 - High-availability compute nodes for LLM inference.
 - GPUs (optional, for Vision models).
 - Secure storage for documents, generated test cases, and logs.

3.3.3 Software Interfaces

1. **Frontend:** Developed in React.js (with Vite) and TailwindCSS, interfacing with the backend via REST APIs.
2. **Backend Services:**

- Node.js (TypeScript) backend to handle business logic and orchestrate FastAPI-based AI services.
- FastAPI (Python) microservices for LLM integration and prompt handling.

3. Databases:

- MongoDB (optional) – for flexible storage of dynamic documents or raw AI output.

4. LLM APIs:

- Integration with OpenAI, HuggingFace, or other hosted LLM endpoints for text and vision processing.

3.3.4 Communication Interfaces

- **REST APIs:** Used for communication between frontend and backend services.
- **gRPC or HTTP (internal):** Used for service-to-service communication between Node.js and Python FastAPI-based microservices.
- **WebSocket (optional):** For real-time updates in chat-based test case generation or feedback handling.
- **HTTPS:** All external communication is secured over HTTPS to ensure data confidentiality and integrity.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system should generate test cases within 3–5 seconds of receiving the input, depending on input complexity.
- It must support concurrent users, handling at least 100 simultaneous sessions without noticeable performance degradation.
- The AI chat interface should provide a response time under 2 seconds for standard queries.

3.4.2 Safety Requirements

- The system must ensure that temporary or partial data from failed uploads or interrupted processes is properly cleaned up.
- Graceful error handling must be implemented to prevent system crashes or loss of user inputs.

3.4.3 Security Requirements

- All data transfers must be encrypted using HTTPS.
- User inputs and generated test cases should be stored securely with proper access control mechanisms.
- The platform should support role-based access control (RBAC) to restrict access to administrative or sensitive features.
- Input sanitization must be enforced to prevent injection attacks or malformed file uploads.

3.4.4 Software Quality Attributes

- **Usability:** The user interface should be intuitive, requiring minimal training to operate.
- **Reliability:** The system should maintain 99.9% uptime with automatic retry mechanisms for failed AI requests.
- **Maintainability:** The architecture should follow modular principles, enabling easy updates to AI models, UI components, or database schema.

- **Scalability:** The system should scale horizontally to accommodate increased workload or users.
- **Portability:** The backend services should be deployable in any cloud or containerized environment using Docker or similar tools.

3.5 System Requirements

3.5.1 Database Requirements

- A PostgreSQL database for structured storage of user data, feedback, and test case records.
- Optional use of MongoDB for unstructured or dynamic data storage (e.g., raw LLM outputs, document parsing results).
- Proper indexing and schema design to support fast retrieval of test cases and historical chat logs.

3.5.2 Software Requirements (Platform Choice)

- **Frontend:** React.js (with Vite), TailwindCSS, and Zustand for state management.
- **Backend:** Node.js with TypeScript for core logic and routing.
- **AI Engine:** FastAPI in Python for integration with LLMs.
- **Database:** PostgreSQL and optionally MongoDB.
- **DevOps:** Docker, GitHub Actions, and AWS EC2/Container Services for deployment.

3.5.3 Hardware Requirements

- **Client-Side:** Any system capable of running a modern web browser (Chrome, Firefox, Edge).
- **Server-Side:**
 - Minimum 4-core CPU, 16 GB RAM for backend services.
 - GPU-enabled node (optional but preferred) for vision model processing.

- Secure storage (S3 or equivalent) for uploaded files and generated test case artifacts.

3.6 Analysis Models: SDLC Model to be Applied

The chosen Software Development Life Cycle (SDLC) model for TestGen.AI is the **Agile Model**.

- The project follows an iterative and incremental development process.
- Functional modules (e.g., file parsing, chat interaction, test case rendering) are developed and tested in sprints.
- Feedback from early users and internal testers is used to guide enhancements in each iteration.
- Agile allows flexible adaptation of AI models and UI changes based on evolving requirements.

CHAPTER 4

SYSTEM DESIGN

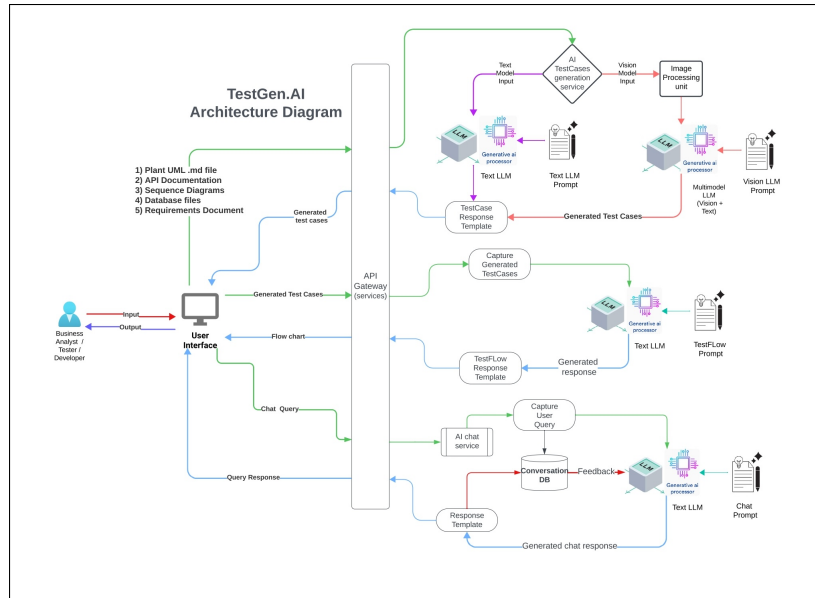


Figure 4.1: Data Flow Diagram

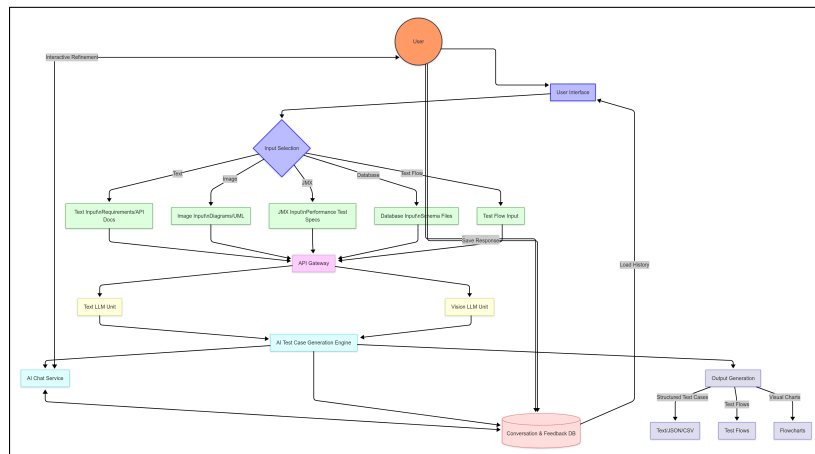


Figure 4.2: Data Flow Diagram

4.3 Entity Relationship Diagram

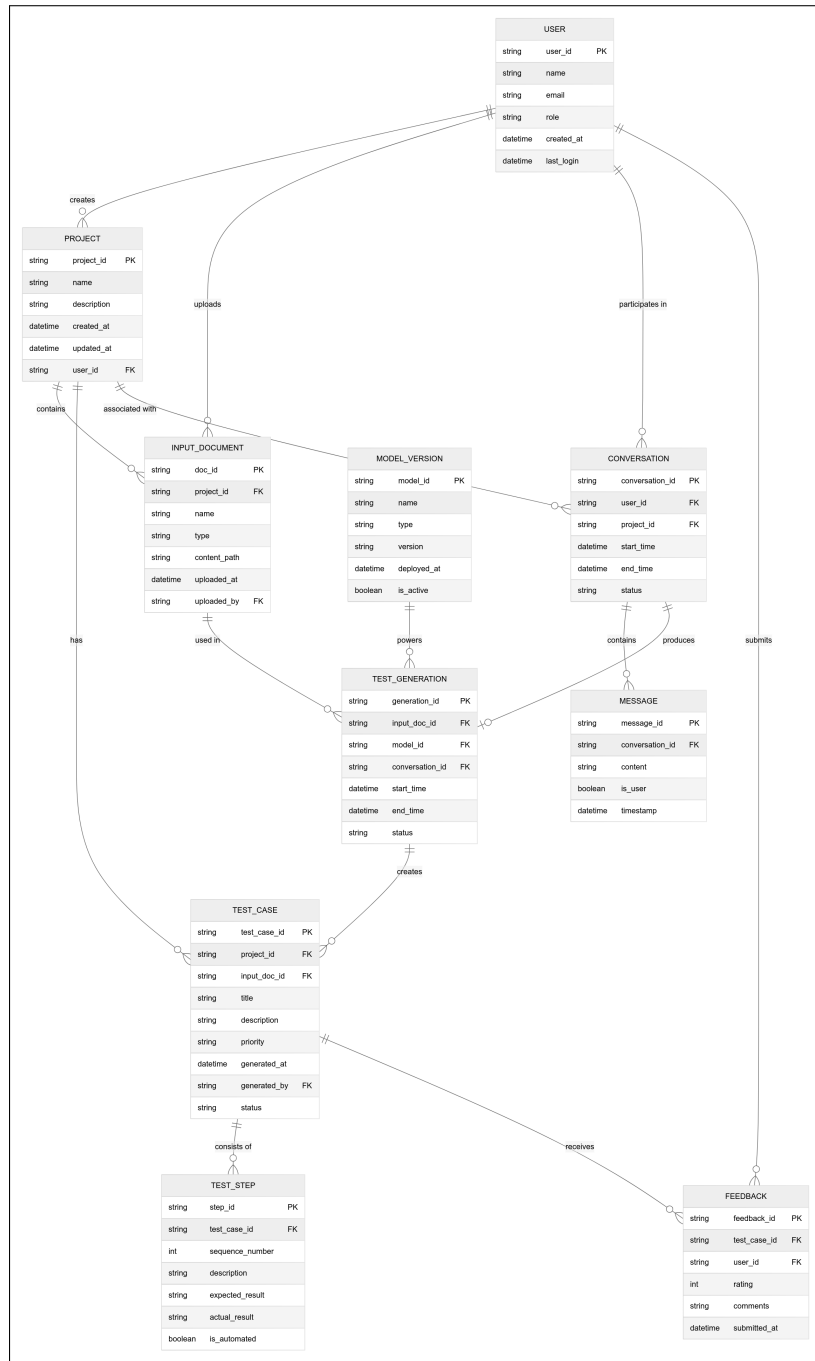


Figure 4.3: ER Diagram

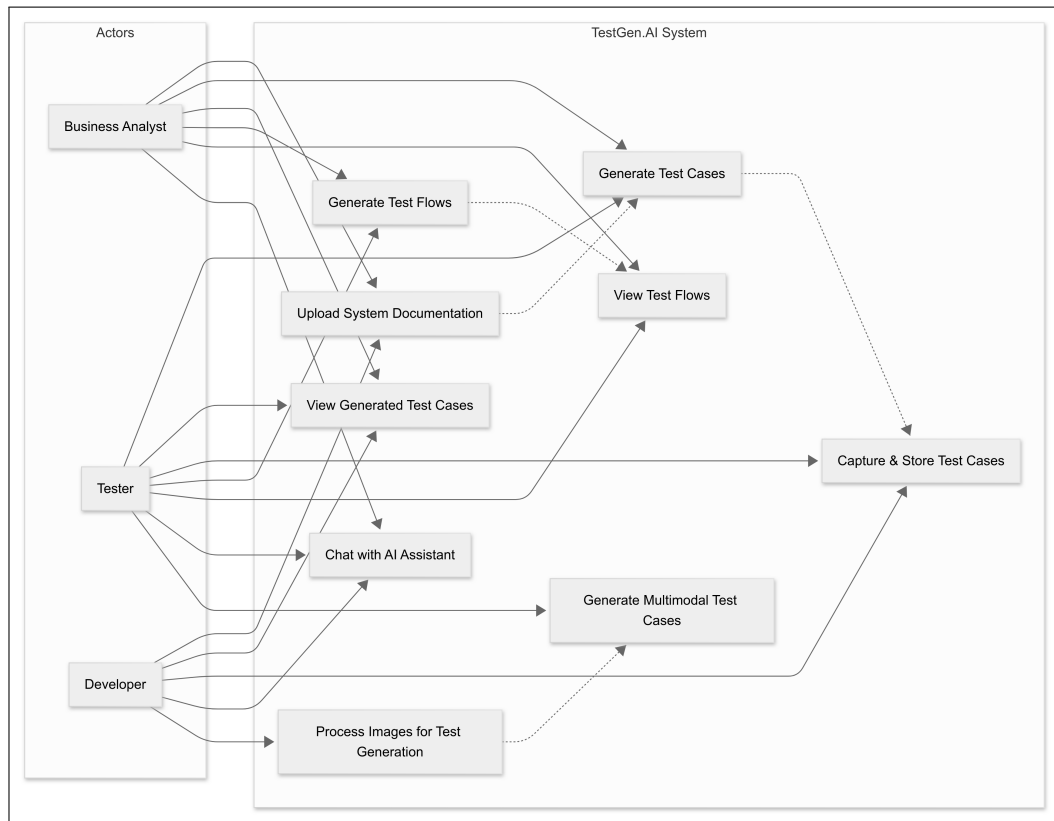
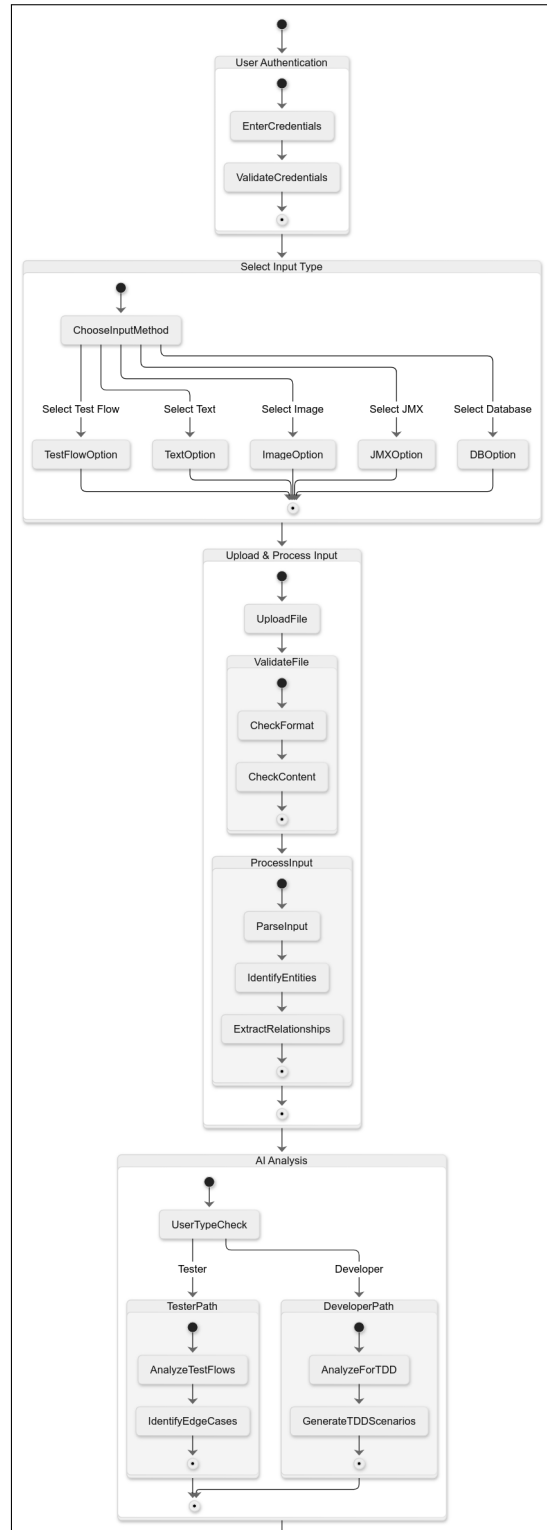


Figure 4.4: Use Case Diagram

4.4 UML Diagrams



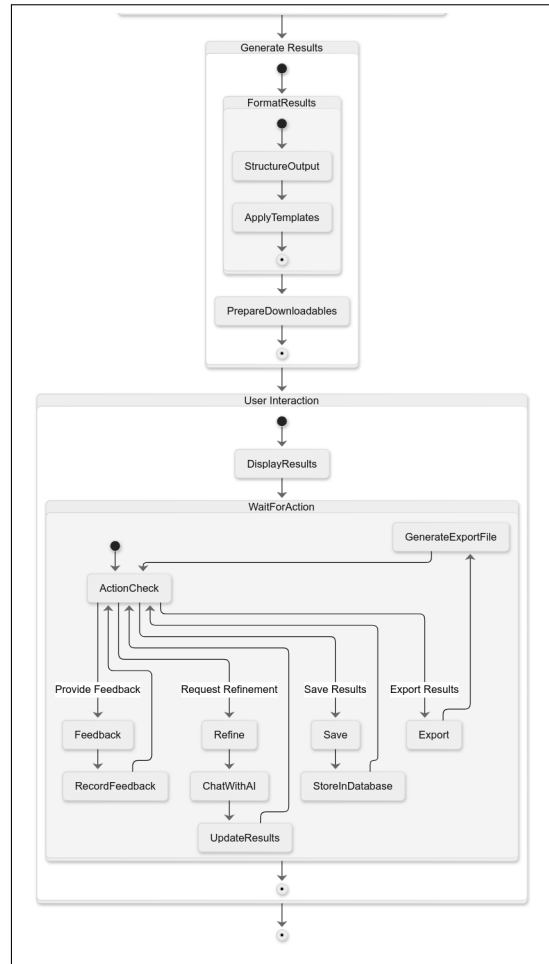


Figure 4.6: Activity Diagram

4.5 Sequence Diagram

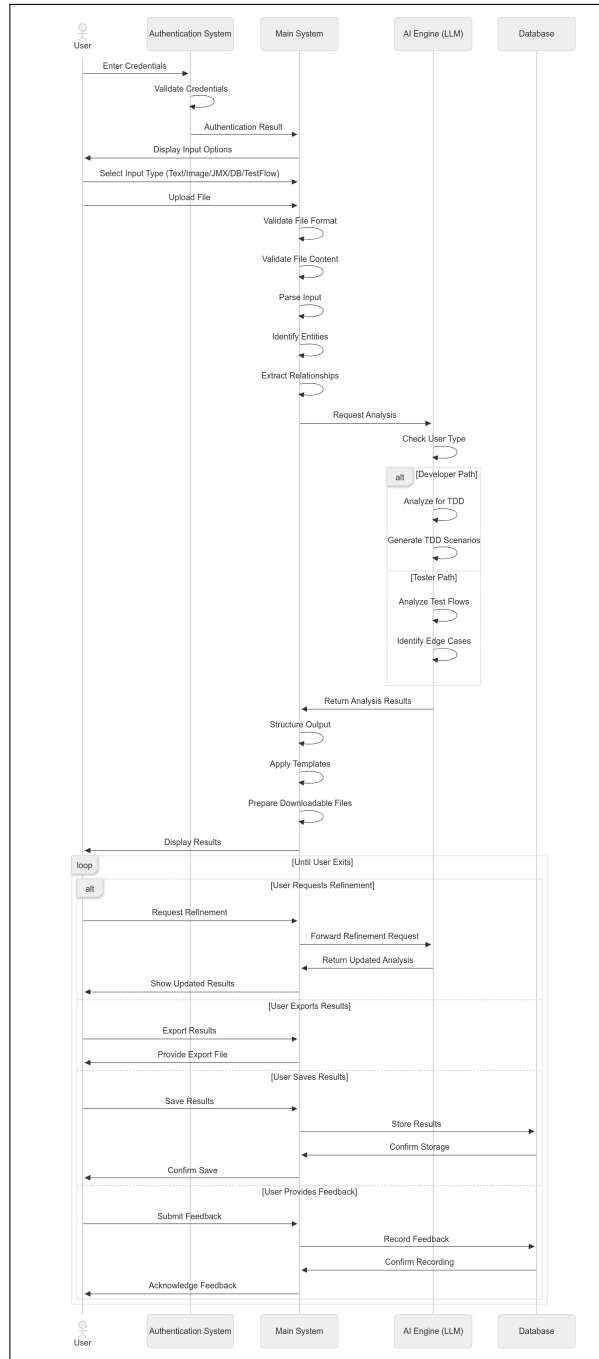


Figure 4.7: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Requirements Finalization and System Design: (Estimated 1 month)** Stakeholder Consultations: Gather inputs from stakeholders to define system functionality and scope. Requirements Specification: Document all functional and non-functional requirements. System Design: Create block diagrams and architecture for modules such as NLP engine, test case generator, and reporting tools. Technology Stack: Finalize tools and frameworks like Python, Flask, AI/ML libraries, and databases.
- **Module Development: (Estimated 2 months)** NLP-based Test Case Generation: Develop AI models to parse user requirements and generate test cases, supporting functional, boundary, and edge cases. Reporting Tool: Create a tool for exporting and viewing generated test cases in formats like JSON or human-readable reports. Integration with Testing Frameworks: Connect generated test cases with CI/CD pipelines for seamless automation.
- **System Integration and Optimization: (Estimated 1 months)** Module Integration: Ensure smooth communication between the NLP engine, test case generator, and reporting tools. Performance Optimization: Improve speed and accuracy, with scalability for handling large documents or multiple users. Customization and Tuning: Enable domain-specific tuning and user-defined test case parameters.
- **User Testing and Feedback Implementation: (Estimated 1 months)** Beta Testing: Deploy the system to users for real-world testing and collect feedback. UX Refinement: Improve the user interface based on feedback and fix identified bugs. Quality Assurance: Conduct thorough testing to ensure system reliability and accuracy.
- **Documentation and Review: (Estimated 1 months)** User Guides and Documentation: Provide comprehensive guides and technical documentation for users and developers. Stakeholder Reviews: Regularly review progress and gather feedback. Publication: Document findings and contribute to relevant AI and testing communities.

Based on these estimates, the total project timeline would be around 15 months. However, it's important to note that these estimates are just an

example and actual timelines may vary based on the specific requirements, constraints of each project, and availability of resources.

5.1.2 Project Resources

1. **Software Developers** Responsible for designing, developing, and optimizing the software and features of the application.
2. **AI Developers** Responsible for development of RAG pipeline for the test case generation from documents.
3. **Project Management Tool:** Jira will be utilized for tracking project tasks, managing workflows, and facilitating communication among team members. It will help in organizing sprints, monitoring progress, and ensuring that the project stays on schedule.
4. **Collaboration Tools:** GitHub for version control and collaborative coding, Google Colab for sharing development notes and prototypes, Google Meet for team meetings and discussions, and Discord for real-time communication and collaboration among team members.

5.2 Risk Management

5.2.1 Risk Identification

- **Technical Risk:**
 1. The complexity of implementing multiplayer functionality could lead to delays or suboptimal performance.
 2. Challenges in optimizing game graphics and performance across various devices.
 3. Difficulties in ensuring seamless real-time synchronization of player actions.
 4. Risk of bugs and issues arising from diverse player interactions during gameplay.
- **Financial Risk:** Budget overruns could occur due to unexpected expenses related to software licenses, hardware upgrades, and development tools.
- **Operational Risk:** Unforeseen changes in project scope or team composition could disrupt the project timeline and deliverables.

5.2.2 Risk Analysis

- **Technical Risk:** This risk is assessed as high impact, as it could significantly hinder the game's performance and development timeline, with a medium likelihood of occurrence. To mitigate this risk, regular performance testing and code reviews are planned.
- **Financial Risk:** The financial risk is evaluated as medium impact, with a low likelihood of occurrence. To mitigate this risk, careful budget tracking and contingency planning are in place to address potential overruns.
- **Operational Risk:** The operational risk is seen as medium impact with a medium likelihood. To mitigate this risk, a clear project scope and change management process will be established to manage any scope changes effectively.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Technical Risk Mitigation:** Regular performance testing and code reviews will be conducted to identify and resolve technical issues early in the development process.
- **Financial Risk Mitigation:** A robust budget tracking system and regular financial reviews will be implemented to ensure the project remains within budget constraints.
- **Operational Risk Mitigation:** A clearly defined project scope and an established change management process will be used to evaluate and manage any potential scope changes thoroughly.
- **Risk Monitoring:** Weekly risk assessment meetings will be held to track the status of identified risks and address any new risks that may arise during the project lifecycle.
- **Risk Management:** A contingency fund of 10% of the total budget has been allocated to address unforeseen issues that may arise throughout the project.

5.3 Project Schedule

5.3.1 Project Task Set

- **Environment Setup:** Set up tools like Python, Flask, and version control (Git). Integrate the Gemini API and install necessary libraries for RAG.
- **Requirement Gathering and Design:** Finalize system requirements through stakeholder meetings. Research RAG techniques and design system architecture with block diagrams.
- **Prototype Development:** Build a prototype to utilize the Gemini API for parsing user requirements. Generate initial test cases in the specified format and evaluate with sample documents.
- **Module Development:** Develop the RAG model to extract information from requirements and generate structured test cases. Implement the reporting tool for formatted test case outputs.
- **System Integration:** Integrate RAG modules and the Gemini API, ensuring smooth communication. Optimize the system for handling large datasets and improving performance.
- **Customization and Tuning:** Enable user-defined customization for test case parameters. Support domain-specific fine-tuning of the RAG model as needed.
- **Testing and Quality Assurance:** Set up automated testing frameworks to evaluate the accuracy of generated test cases. Identify and resolve any bugs related to RAG and integration.
- **User Testing and Feedback Collection:** Deploy the system for real-world user testing with the Gemini API. Gather feedback and refine the system based on user input.
- **Documentation and Publication:** Document the development process and create user manuals. Publish findings in relevant AI/ML and software testing communities.
- **Final Review and Deployment:** Conduct final reviews with stakeholders. Prepare the system for deployment in CI/CD pipelines.

5.3.2 Timeline Chart

Figure 5.1 is the summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

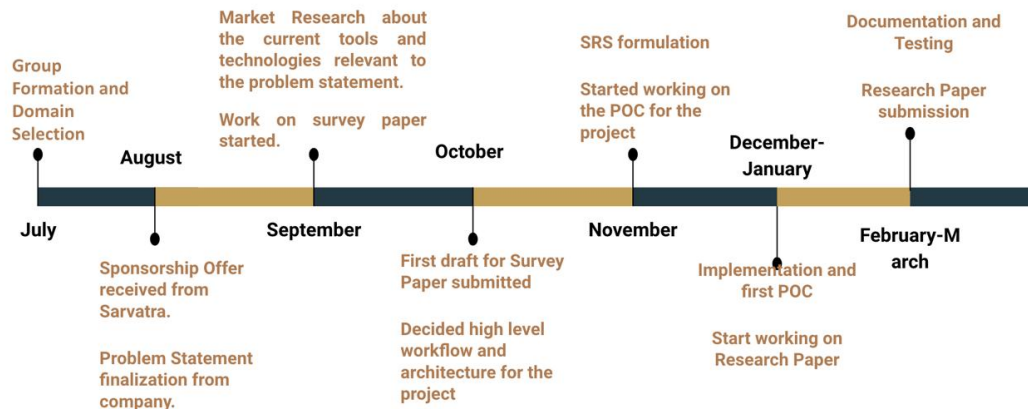


Figure 5.1: Summarized Timeline Chart

5.4 Team Organization

5.4.1 Team Structure

- **Ayush Bulbule:** Full Stack Developer, DevOps.
- **Soumya Garg:** Full stack, AI Developer, Backend Design.
- **Awadhoot Khutwad:** Designer, AI Developer, Full Stack Developer.
- **Sumit Suryawanshi:** AI Developer, Devops.

5.4.2 Management Reporting and Communication

- **Prof. M. S. Wakode:** Internal mentor and guide.
- **Nikhil Karkhanis:** External mentor and guide.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **User Interface Module** Acts as the main interaction point for Business Analysts, Testers, and Developers. It accepts various forms of input such as Plant UML ‘.md’ files, API Documentation, Sequence Diagrams, Database files, and Requirements Documents. It generates Test Cases, Flow Charts, and Query Responses, handles chat queries, and displays the generated outputs.
- **API Gateway (Services)** This module orchestrates communication between the User Interface and other internal services. It manages test case generation requests, TestFlow responses, AI chat service queries, and routes the outputs back to the User Interface.
- **AI Test Cases Generation Service** This service utilizes Text LLM for processing textual information and Multimodal LLM (Vision + Text) for handling image-based test cases through an Image Processing Unit. It generates test cases from provided documentation and vision-based test cases for UI components or visual elements.
- **Image Processing Unit** This unit processes visual elements like Sequence Diagrams and Database Schemas. It translates image-based inputs into structured test cases using the Multimodal LLM.
- **TestCase and TestFlow Response Templates** These templates format the generated test cases and flow responses and capture the output for seamless integration into reporting and documentation.
- **Capture Modules** Capture Modules are responsible for collecting test cases generated by LLM processors and managing the conversational context by storing queries in the Conversation DB.
- **AI Chat Service** This service responds to user queries based on the Chat Prompt. It generates conversational responses and maintains feedback through the Conversation DB.
- **Conversation DB** The Conversation DB stores user queries, AI-generated responses, and feedback loops for continuous improvement.

6.2 Tools and Technologies Used

- Python

- Nodejs
- Django Rest Framework
- Docker
- LLM
- Git
- GitHub & GitHub Actions
- FastAPI
- Reactjs
- Tailwind css
- MongoDB
- React Flow
- Postman

6.3 Algorithm Details

6.3.1 Algorithm 1: Test Case Generation Using LLMs

The primary algorithm for test case generation leverages Large Language Models (LLMs) such as Gemini, Llama, and Mixtral. It accepts user inputs in various formats like:

- Text-based requirements
- Plant UML ‘.md‘ files
- API Documentation
- Sequence Diagrams
- Database Schemas

The workflow is as follows:

1. **Prompt Construction:** A structured prompt is created that describes the test scenario and expected outcomes.

2. **Model Selection:** Depending on the user's choice, the appropriate LLM model (Gemini, Llama, Mixtral) is selected.
3. **Content Generation:** The LLM generates test cases, including:
 - Test Case ID
 - Test Scenario
 - Pre-conditions and Post-conditions
 - Step-by-step Execution
 - Expected Results
4. **Response Parsing:** The JSON response is parsed and mapped to the data model defined in the `TestCasesContainer`.
5. **Validation and Storage:** Generated test cases are validated and stored for further analysis or execution.
6. **Pseudocode for Algorithm 1: Prompt-Based Test Case Generation**

```
func GeneratePromptBasedTestCases(prompt, model) {  
    // Send the prompt to the LLM model  
    response, err := CallLLMModel(prompt, model)  
    if err != nil {  
        Error("Failed to call LLM model: %v", err)  
        return  
    }  
  
    // Parse the response to extract test cases  
    testCases, err := ParseTestCases(response)  
    if err != nil {  
        Error("Failed to parse test cases: %v", err)  
        return  
    }  
  
    // Structure the test cases and return the result  
    formattedTestCases := StructureTestCases(testCases)  
    return formattedTestCases  
}
```

6.3.2 Algorithm 2: Flow Chart Generation Using Gemini LLM

This algorithm is responsible for converting test cases into interactive flow charts using the ReactFlow framework. The steps involved are:

1. **Input Parsing:** Test cases generated by Algorithm 1 are parsed to extract logical flows and dependencies.
2. **Flowchart Mapping:** Each test case is converted into nodes, and logical transitions are represented as edges.
3. **ReactFlow Integration:** The structured nodes and edges are formatted to comply with ReactFlow's syntax.
4. **Node Arrangement:** Nodes are placed on a grid with proper alignment to depict the logical order of execution.
5. **Error Handling:** Cyclic dependencies and unreachable nodes are flagged for correction.

The final output is an interactive flowchart that visually represents the end-to-end flow of test cases.

6. Pseudocode for Algorithm 2: Code-Based Test Case Generation

```
func GenerateCodeBasedTestCases(codeSnippet, model) {
    // Extract logic paths and conditions from the code
    snippet
    logicPaths, err := ExtractLogicPaths(codeSnippet)
    if err != nil {
        Error("Failed to extract logic paths: %v", err)
        return
    }

    // For each path, generate a test case using the LLM
    for path in logicPaths {
        testCase, err := CallLLMModel(path, model)
        if err != nil {
            Error("Failed to generate test case for path: %v", err)
            continue
        }
        testCases.append(testCase)
    }
}
```



```
    }

    // Structure and format the test cases
    formattedTestCases := StructureTestCases(testCases)
    return formattedTestCases
}
```

6.3.3 Algorithm 3: Vision-Based Test Case Generation

This algorithm extends the capabilities of LLMs to image-based artifacts such as:

- Sequence Diagrams
- Database Schemas
- UI Mockups

The steps are as follows:

1. **Image Encoding:** The input image is converted to Base64 encoding for processing.
2. **Vision Model Invocation:** Depending on the selected model (Gemini Vision or Llama Vision), the encoded image and prompt are sent for analysis.
3. **Contextual Understanding:** The LLM interprets relationships, actions, and triggers in the image to derive test cases.
4. **Test Case Structuring:** Similar to text-based generation, the derived cases are structured with:
 - Test steps extracted from visual cues
 - Expected behavior based on diagram flow
 - Edge cases inferred from branching paths
5. **Output Generation:** The resulting test cases are formatted in JSON, ready for integration with the test suite.
6. **Pseudocode for Algorithm 3: Vision-Based Test Case Generation**

```
func GenerateVisionBasedTestCases(imageFile, model, prompt)
{
    // Step 1: Encode the image to Base64 format
    encodedImage, err := EncodeImageToBase64(imageFile)
    if err != nil {
        Error("Failed to encode image: %v", err)
        return
    }

    // Step 2: Call the selected vision model (Gemini or Llama)
    response, err := CallVisionModel(encodedImage, model,
        prompt)
    if err != nil {
        Error("Vision model invocation failed: %v", err)
        return
    }

    // Step 3: Parse the response to understand the context
    testCases, err := ParseVisionResponse(response)
    if err != nil {
        Error("Failed to parse vision response: %v", err)
        return
    }

    // Step 4: Structure the test cases with test steps
    formattedTestCases := StructureTestCases(testCases)

    // Step 5: Return the test cases in JSON format
    return formattedTestCases
}
```

6.3.4 Algorithm 4: Conversational Agent for Test Case Querying

This algorithm enables interactive querying and refinement of generated test cases using a conversational agent. The main steps are:

1. **Conversation Context:** The user's conversation history is maintained for context-awareness.
2. **User Query Processing:** Queries about specific test cases, edge conditions, or flow details are processed.

3. **Prompt Engineering:** A structured prompt is created, including the conversation history, to query the LLM.
4. **Response Generation:** The model responds with explanations, edge cases, or suggested modifications to the test case.
5. **Feedback Loop:** User feedback is incorporated for continuous learning and context enrichment. This conversational interface allows for real-time exploration and understanding of test scenarios, ensuring that all possible test paths are considered.
6. **Pseudocode for Algorithm 4: Conversational Agent for Test Case Querying**

```
func
ConversationalAgentForTestCaseQuery(conversationHistory,
    query, model) {
    // Step 1: Maintain the conversation history for context
    context := MaintainConversationContext(
        conversationHistory)

    // Step 2: Process the user's query
    structuredPrompt := CreateStructuredPrompt(context,
        query)

    // Step 3: Call the LLM with the structured prompt
    response, err := CallLLMModel(structuredPrompt, model)
    if err != nil {
        Error("Failed to query LLM model: %v", err)
        return
    }

    // Step 4: Generate explanations or modifications based
    on the query
    explanation, err := ParseResponse(response)
    if err != nil {
        Error("Failed to parse LLM response: %v", err)
        return
    }

    // Step 5: Send the response back to the user and update
    context
    updatedContext := UpdateContextWithResponse(context,
        explanation)
```

```
// Step 6: Return the updated conversation and
    explanations
return updatedContext, explanation
}
```

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

7.1.1 Backend Testing

- **Unit Testing with Postman:** Postman was used for unit testing the backend APIs built with **FastAPI**. We created and executed test scripts to validate individual endpoints for the following algorithms:
 - *Text-Based Test Case Generation*
 - *Vision-Based Test Case Generation*
 - *Conversational Agent for Test Case Querying*

Each API was tested for:

- Correct response status codes (200, 404, 500, etc.)
- Proper data structure and format in the responses
- Edge cases like invalid inputs, missing parameters, and empty payloads

A dedicated Postman collection was maintained to streamline testing and facilitate quick verification for developers.

- **Auto Schema Testing with Swagger (FastAPI Docs):** Swagger documentation was automatically generated through FastAPI's in-built `/docs` endpoint. This allowed for:
 - Real-time visualization of API specifications and models
 - Interactive testing of API calls directly from the browser
 - Validation of request and response data against OpenAPI specifications

Swagger's interactive interface made it easy for developers to understand endpoint behavior and perform quick checks without switching tools.

- **Interactive Testing with FastAPI UI:** FastAPI provides an auto-generated UI for testing APIs in real-time. This UI was leveraged to:
 - Verify endpoint accessibility and parameter passing
 - Check response payloads for correctness
 - Perform quick debugging during development

This interactive interface reduced the dependency on external tools and made local testing seamless.

7.1.2 Frontend Testing

- **Manual Testing:** Manual testing was conducted to explore the user interface and interactive features of the test case generation platform. Different test scenarios included:
 - Generating test cases from varied prompts and code snippets
 - Uploading and processing sequence diagrams, database schemas, and UI mockups for vision-based test generation
 - Engaging with the conversational agent to query and refine generated test cases

These tests ensured that the frontend components were responsive, intuitive, and aligned with the backend logic. All edge cases and UI bugs were addressed during this phase.

7.1.3 Integration Testing

- **End-to-End Testing:** Integration tests were performed to validate the complete flow:
 - Prompt submission → Test case generation → Result display
 - Image upload → Vision-based analysis → Test case generation
 - Conversational agent query → Test case refinement → Display updates

These tests verified that all components communicated correctly, with data flowing seamlessly from frontend to backend and vice versa.

The combination of backend testing with **FastAPI**, **Swagger**, and **Postman**, along with frontend manual testing and integration tests, provided robust validation for the entire system.

7.2 Test cases & Test Results

- Test cases & Test Results

CHAPTER 8

RESULTS

8.1 Outcomes

1. **Automated Test Case Generation:** The system successfully automated the generation of test cases for:
 - Prompt-based descriptions
 - Code snippets
 - Visual artifacts such as sequence diagrams, database schemas, and UI mockups

This automation significantly reduces manual effort and accelerates the testing cycle.

2. **Enhanced Test Coverage:** The platform's ability to automatically derive test cases from both textual and visual inputs ensures comprehensive test coverage, catching edge cases and unhandled scenarios that might be missed in manual testing.
3. **Real-Time Conversational Querying:** Through the interactive conversational agent, testers and developers can query and refine generated test cases in real time. This facilitates quick adjustments, better understanding of edge cases, and smoother debugging during development.
4. **Improved Consistency and Accuracy:** Leveraging LLMs for structured test generation reduces human errors and maintains consistent test case formatting. This ensures alignment with predefined specifications and smooth integration with CI/CD pipelines.
5. **Time and Cost Efficiency:** Automating test case generation from diverse input sources (prompts, code, images) reduces the time required for manual test creation. This, in turn, minimizes development costs and accelerates the software release process.
6. **Scalable and Extensible Architecture:** The system's modular design allows for easy extension, making it adaptable for future enhancements like multi-language support, additional LLM integrations, and deeper code analysis capabilities.
7. **Developer and Tester Empowerment:** The platform empowers developers and testers by providing an intuitive interface for generating, refining, and executing test cases, boosting productivity and confidence in application reliability.

8.2 Screenshots

8.2.1 Text Input and Output

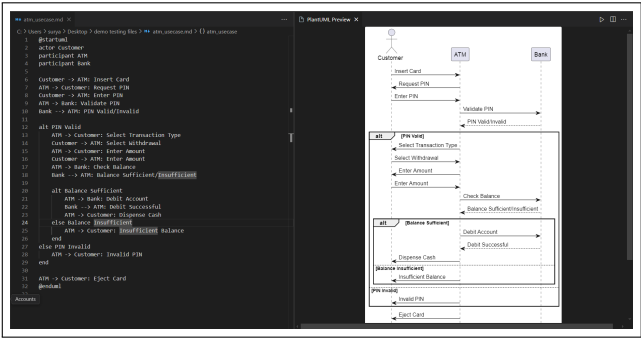


Figure 8.1: Input .md File ATM

ID	Test Scenario	Description	Actions
TC_ATM_002	Invalid PIN Entry	Enter an incorrect PIN.	
TC_ATM_003	Insufficient Balance Withdrawal Attempt	Attempt to withdraw an amount greater than the available bal...	
TC_ATM_001	Valid PIN and Sufficient Balance Withdrawal	Withdraw cash with valid PIN and sufficient balance.	
TC_ATM_006	Eject Card without Transaction	Eject card without performing any transaction after entering P...	
TC_ATM_007	ATM Session Timeout	Do not perform any action after inserting card and entering PI...	
TC_ATM_008	Cancelled Transaction	Cancel the withdrawal transaction after entering the amount.	
TC_ATM_009	Check Balance Inquiry	Check available balance with valid PIN.	
TC_ATM_010	Fast Cash Withdrawal	Withdraw cash using test cash option with valid PIN and suff...	
TC_ATM_005	Maximum Withdrawal Limit	Attempt to withdraw amount greater than maximum allowed li...	
TC_ATM_004	Error Test case description...		

Figure 8.2: Generated Test Cases

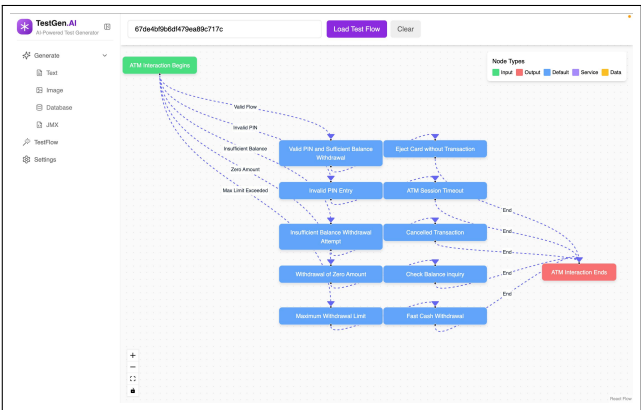


Figure 8.3: Test Flow

8.2.2 Image Input and Output

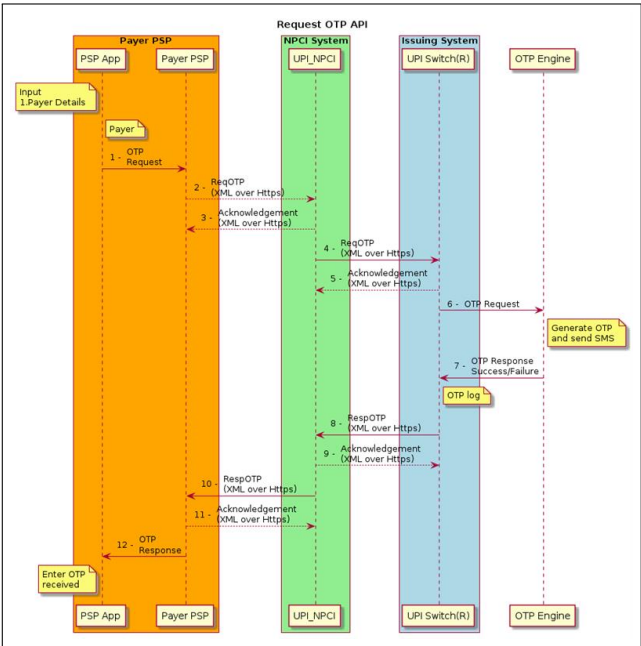


Figure 8.4: Input Image 3

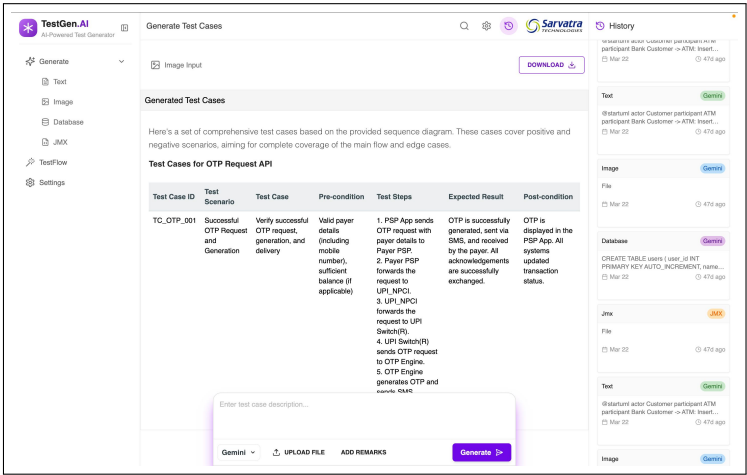


Figure 8.5: Output Image 3a

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

1. The developed platform successfully automates the generation of test cases from both text-based and vision-based artifacts, significantly reducing manual effort in the software testing process.
2. The integration of LLMs for natural language test case generation and vision-based analysis improves the accuracy and coverage of test cases, ensuring a robust testing mechanism.
3. Real-time conversational querying enhances interactive exploration of test scenarios, enabling deeper understanding and efficient error detection.
4. The automated test generation and querying features streamline the overall testing pipeline, leading to faster software release cycles and improved product quality.

9.2 Future Work

1. Incorporation of more sophisticated vision models to handle complex diagrams like ER models, architecture designs, and more.
2. Extending the conversational agent capabilities to suggest optimized test cases based on historical testing data and failure patterns.
3. Integration with CI/CD pipelines for seamless automated testing during each stage of development.
4. Exploring multi-language support for test case generation, making it accessible for global development teams.
5. Adding self-healing mechanisms to automatically adapt to minor changes in UI or backend endpoints during testing.

9.3 Applications

1. **Software Development Lifecycle (SDLC):** Enhancing test coverage during development and deployment stages.
2. **Continuous Integration/Continuous Deployment (CI/CD):** Automated test generation and execution, ensuring rapid and reliable deployments.
3. **Web Application Testing:** Automated UI and API testing to detect vulnerabilities and functional issues early.
4. **Machine Learning Model Validation:** Ensuring model endpoints and data pipelines are thoroughly tested before production.
5. **Database Schema Validation:** Automatically validating schema consistency and data integrity during schema changes.
6. **Cross-Platform Application Testing:** Supporting test case generation for mobile, desktop, and cloud-based applications.

CHAPTER 10

APPENDIX

10.1 Appendix A: Feasibility Study

10.1.1 Technical Feasibility

- The project utilizes modern web technologies such as **FastAPI** for backend development and **Swagger** for API documentation and testing, ensuring technical robustness and scalability.
- Integration with **LLMs (Large Language Models)** like Gemini Vision or Llama Vision for test case generation from both text-based and vision-based artifacts demonstrates advanced technical capability.
- Real-time **WebSocket-based communication** allows seamless interaction for file operations and conversational querying, enhancing user experience.
- The system architecture is designed to support high concurrency and real-time processing, making it suitable for large-scale enterprise applications.
- The project is containerized using **Docker** for portability and isolated testing environments, ensuring consistent behavior across different platforms.

10.1.2 Operational Feasibility

- User testing and feedback from developers and testers indicate the platform is highly operational, with smooth test case generation and real-time query handling.
- The vision-based test case generation module efficiently processes sequence diagrams, database schemas, and UI mockups, significantly reducing manual effort.
- The interactive querying module allows users to refine and validate test cases seamlessly, improving overall testing efficiency.
- Real-time file operations, including reading, writing, and folder management, are handled effectively through WebSocket communication, ensuring smooth backend operations.
- The platform's user-friendly interface supports easy navigation and interaction, making it accessible for both technical and non-technical users.

10.1.3 Economic Feasibility

- A thorough cost-benefit analysis was conducted to assess the economic feasibility of the platform.
- The costs include platform development, server hosting, LLM API costs, and maintenance. However, these are optimized using Docker-based deployments and scalable architecture.
- The benefits, such as automated test case generation, real-time querying, and streamlined backend operations, lead to reduced manual testing efforts and faster deployment cycles.
- Enhanced efficiency in test case generation and validation reduces time-to-market for software projects, translating into significant cost savings.
- The analysis indicated that the return on investment (ROI) is substantial, validating the platform's economic feasibility.

10.2 Appendix B

10.2.1 Survey Paper: Conference Information

- **Conference Title:** International Conference on Emerging Research in Computing, Information, Communication, Artificial Intelligence, and Machine Learning (ERCICAM 2025)
- **Conference Dates:** 7th March 2025
- **Conference Website:** <https://nmit.ac.in/ercicam/>
- **Paper Status:** Accepted

10.2.1.1 Accepted Paper Details

- **Paper Title:** Test Case Generation from User Requirements using LLM Generative AI : A Survey
- **Presentation Type:** Oral Presentation

10.2.1.2 Accepted Paper Certificate



Figure 10.1: Certificate of Authorship – ERCICAM 2025 (Scopus Indexed)

10.2.2 Implementation Paper: Conference Information

- **Conference Title:**2nd IEEE International Conference on Integration of Computational Intelligent System (ICICIS-2025)
- **Submission Status:** Submitted
- **Paper Title:** Test Case Generation from User Requirements using LLM Generative AI
- **Conference Website:** <https://ieee-icicis.in/>

10.2.2.1 Paper Submission Details

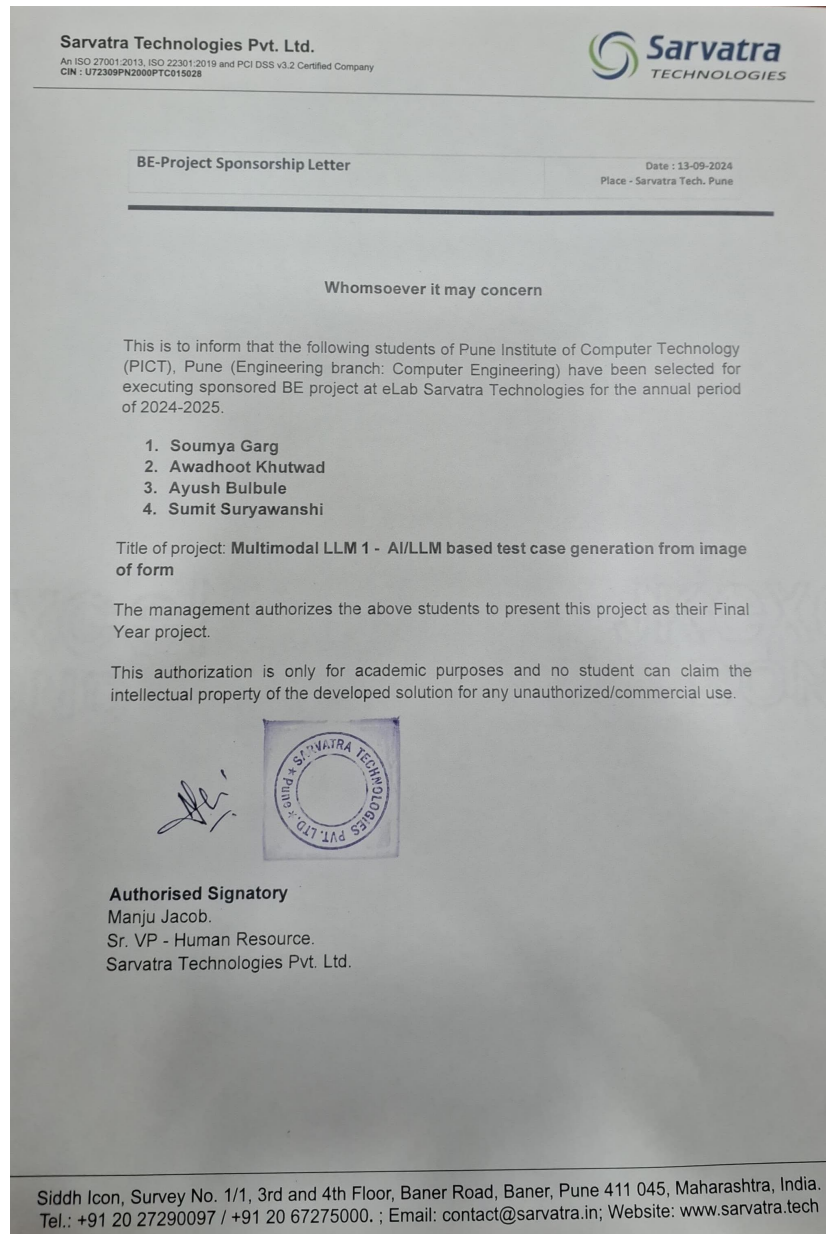
+ Create new submission...				
1 - 1 of 1				
Show 25 50 100 All Clear All Filters				
Paper ID	Title	Track	Files	Actions
471	Test Case Generation from User Requirements using LLM & Generative AI (Show details)	Track 04-Artificial & Computational Intelligence	Submission files: ① Test Case Generation from User Requirements using LLM & Generative AI.pdf	Submission: 🔗 Edit Submission 🔗 Edit Conflicts ✕ Delete Submission Supplementary Material: 🔗 Upload Supplementary Material

Figure 10.2: Screenshot of the Paper Submission

10.3 Appendix C

10.3.1 Sponsorship Certificate

This project has been sponsored by Sarvatra Technologies. The Sponsorship Certificate is provided below for reference.



Sarvatra Technologies Pvt. Ltd.
An ISO 27001:2013, ISO 22301:2019 and PCI DSS v3.2 Certified Company
CIN : U72309PN2000PTC015028

Sarvatra TECHNOLOGIES

BE-Project Sponsorship Letter

Date : 13-09-2024
Place - Sarvatra Tech. Pune

Whomsoever it may concern

This is to inform that the following students of Pune Institute of Computer Technology (PICT), Pune (Engineering branch: Computer Engineering) have been selected for executing sponsored BE project at eLab Sarvatra Technologies for the annual period of 2024-2025.

1. Soumya Garg
2. Awadhoot Khutwad
3. Ayush Bulbule
4. Sumit Suryawanshi

Title of project: **Multimodal LLM 1 - AI/LLM based test case generation from image of form**

The management authorizes the above students to present this project as their Final Year project.

This authorization is only for academic purposes and no student can claim the intellectual property of the developed solution for any unauthorized/commercial use.




Authorised Signatory
Manju Jacob.
Sr. VP - Human Resource.
Sarvatra Technologies Pvt. Ltd.

Siddh Icon, Survey No. 1/1, 3rd and 4th Floor, Baner Road, Baner, Pune 411 045, Maharashtra, India.
Tel.: +91 20 27290097 / +91 20 67275000. ; Email: contact@sarvatra.in; Website: www.sarvatra.tech

Figure 10.3: Sponsorship Certificate from Sarvatra Technologies

10.4 Appendix D

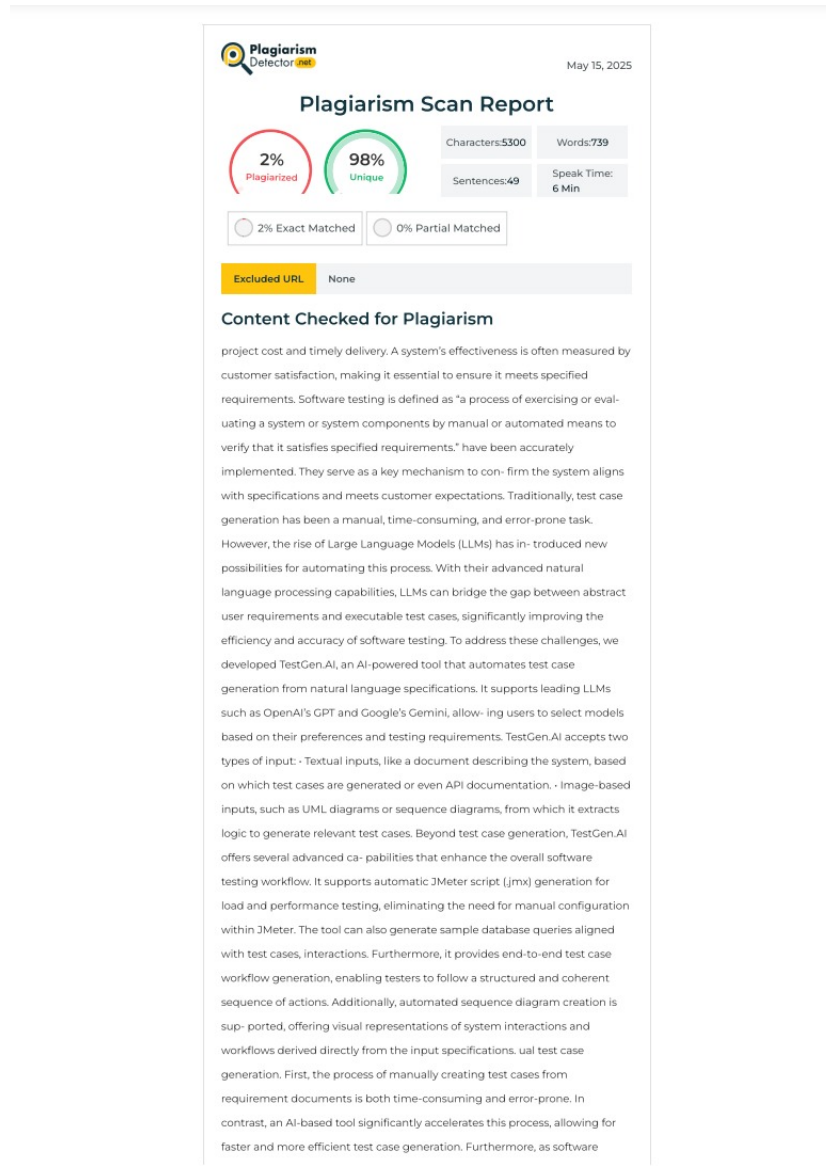


Figure 10.4: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Arora, Chetan, Tomas Herda, and Verena Homm. "Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study." arXiv preprint arXiv:2404.12772 (2024). Chetan Arora, Tomas Herda, Verena Homm
- [2] Sami, Abdul Malik, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Herda Tomas, and Pekka Abrahamsson. "A Tool for Test Case Scenarios Generation Using Large Language Models." arXiv preprint arXiv:2406.07021 (2024).
- [3] Alshahwan, Nadia, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. "Automated unit test improvement using large language models at meta." In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, pp. 185-196. 2024.
- [4] Li, Kefan, and Yuan Yuan. "Large Language Models as Test Case Generators: Performance Evaluation and Enhancement." arXiv preprint arXiv:2404.13340 (2024).
- [5] Steenhoek, Benjamin, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. "Reinforcement learning from automatic feedback for high-quality unit test generation." arXiv preprint arXiv:2310.02368 (2023).
- [6] ISO, ISO. "IEC/IEEE International Standard-Systems and software engineering-System life cycle processes." ISO/IEC/IEEE 15288 First edition 2015-05-15, Technical report (2015).
- [7] Sami, Abdul Malik, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Herda Tomas, and Pekka Abrahamsson. "A Tool for Test Case Scenarios Generation Using Large Language Models." arXiv preprint arXiv:2406.07021 (2024).
- [8] Malik Sami, Abdul, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Herda Tomas, and Pekka Abrahamsson. "A Tool for Test Case Scenarios Generation Using Large Language Models." arXiv e-prints (2024): arXiv-2406.
- [9] Wang, Junjie, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. "Software testing with large language models: Survey, landscape, and vision." IEEE Transactions on Software Engineering (2024).

- [10] Myers, Glenford J., Corey Sandler, and Tom Badgett. The art of software testing. John Wiley and Sons, 2011.
- [11] Pezze, M. Software testing and analysis: process, principles, and techniques. John Wiley and Sons, 2008.
- [12] Anwar, Nahid, and Susmita Kar. "Review paper on various software testing techniques and strategies." Global Journal of Computer Science and Technology 19.2 (2019): 43-49.
- [13] Siniaalto, Maria, and Pekka Abrahamsson. "A comparative case study on the impact of test-driven development on program design and test coverage." First International Symposium on Empirical Software Engineering and Measurement (ESEM 2007). IEEE, 2007.
- [14] Alshahwan, Nadia, et al. "Automated unit test improvement using large language models at meta." Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024.
- [15] Minaee, Shervin, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. "Large language models: A survey." arXiv preprint arXiv:2402.06196 (2024).
- [16] Hurani, Muaz, and Hamzeh Idris. "Investigating the use of LLMs for automated test generation: challenges, benefits, and suitability." (2024).
- [17] Bhatia, Shreya, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. "Unit test generation using generative ai: A comparative performance analysis of autogeneration tools." In Proceedings of the 1st International Workshop on Large Language Models for Code, pp. 54-61. 2024.
- [18] Dakhel, Arghavan Moradi, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. "Effective test generation using pre-trained large language models and mutation testing." Information and Software Technology 171 (2024): 107468.
- [19] Fan, Angela, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. "Large language models for software engineering: Survey and open problems." In 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), pp. 31-53. IEEE, 2023.

- [20] Ma, W., S. Liu, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, and Y. Liu. "The Scope of ChatGPT in Software Engineering: A Thorough Investigation. arXiv (2023)."